SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – ARCHITECTURE / MODELING (SECURE INFRASTRUCTURE)

Course Code:	CSA5802	**Course Title:**	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO4 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	Design secure DevSecOps infrastructure by developing architecture diagrams, modelling deployment workflows, Identity and Access Management (IAM) relationships, and GitOps workflows.		
Student Name:	Krithika M	**Reg. No.:**	192521385
Section / Batch:	B-Tech IT	**Date:**	24 08 26

Instructions to Students

• Design all architecture diagrams, deployment workflows, IAM relationships, and GitOps workflows originally and clearly.
• Use DiagrammeR or Draw.io for architecture modelling and maintain consistent labels, trust boundaries, data flows, and security controls.
• Clearly represent IAM roles/permissions, Infrastructure as Code (IaC), GitOps flow, and DevSecOps security integration wherever applicable.
• Include editable source files for diagrams, IaC/configuration files, methodology documentation, and all supporting project artifacts.
• Submit the GitHub repository link and final PDF report through Google Classroom before the specified deadline; verify that the repository is accessible and complete.

Question Type	Focus Area	Questions	Marks
ARCHITECTURE / MODELING	Design and Application Based	Q1	Q1 – 100
	GRAND TOTAL:	**100 Marks**	

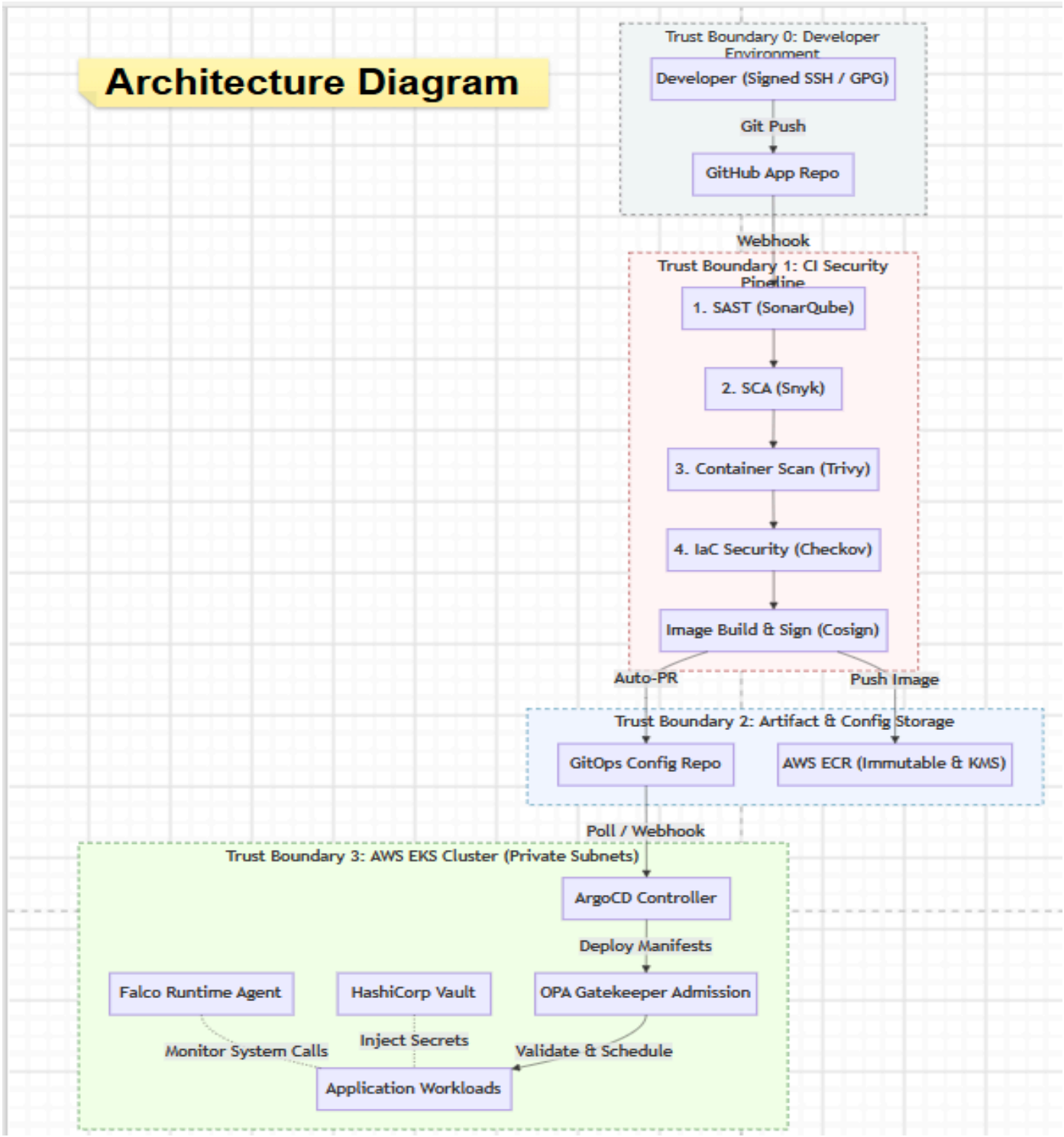
RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
IAM Model and Access Control Design	20%	Designs a comprehensive IAM model with clearly defined identities, roles, permissions, authentication and least-privilege access controls	Designs an appropriate IAM model with most roles and access controls correctly represented and only minor gaps	Provides a basic IAM model with limited role definition, permissions or access-control details	Provides an incomplete or inaccurate IAM model with weak or inappropriate access controls
Infrastructure as Code (IaC) Modelling	30%	Accurately models infrastructure provisioning through IaC with reusable, consistent and secure configuration components	Models IaC workflow correctly with appropriate configuration structure and only minor omissions	Provides a basic IaC model with limited automation, structure or security considerations	Provides an incomplete or incorrect IaC model with poor configuration and automation representation
GitOps Workflow and Version Control	20%	Clearly models a complete GitOps workflow including repository changes, review, automated synchronization, deployment and desired-state management	Models the GitOps workflow correctly with most stages represented and minor workflow gaps	Provides a basic GitOps workflow with missing stages or unclear repository-to-deployment relationships	Provides an incomplete or inaccurate GitOps workflow with weak version-control integration
Security Integration and Workflow Automation	20%	Effectively integrates IAM, IaC and GitOps with security validation, approvals, automation and auditable workflow controls	Integrates appropriate security and automation controls with only minor gaps	Shows basic security integration and automation but several controls or workflow relationships are missing	Shows limited or incorrect integration of security controls and workflow automation
Documentation and Workflow Clarity	10%	Clearly documents IAM, IaC and GitOps components and presents the complete workflow with strong technical justification	Provides clear documentation and workflow explanation with minor clarity or justification gaps	Provides basic documentation with limited explanation of workflow components and relationships	Provides incomplete or unclear documentation with little technical justification

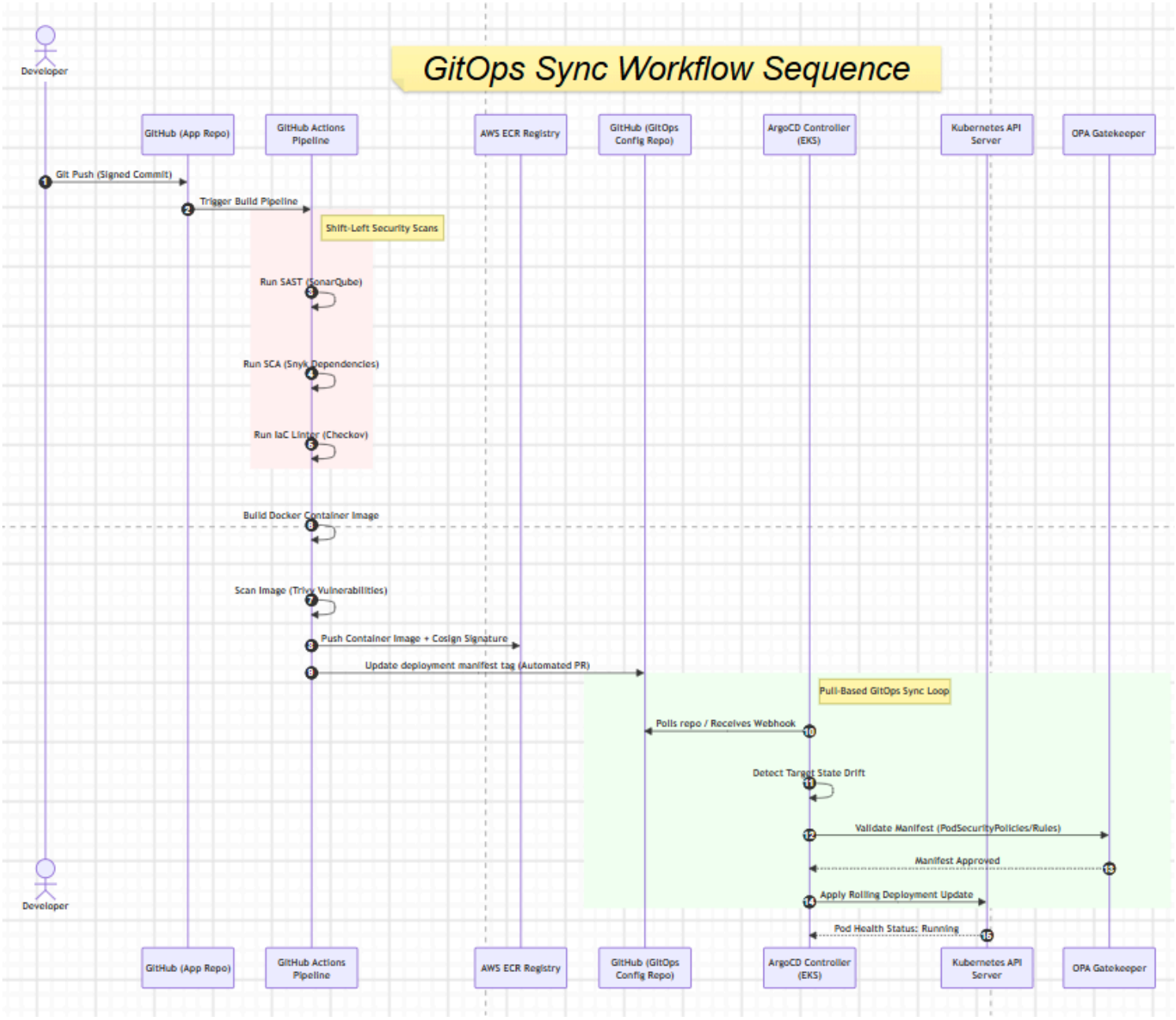
Rubrics:

Criteria	Weightage	Q5 (10)	Total Marks (50)
IAM Model and Access Control Design	25%		
Infrastructure as Code (IaC) Modelling	30%		
GitOps Workflow and Version Control	20%		
Security Integration and Workflow Automation	15%		
Documentation and Workflow Clarity	10%		
Total per Question	**100 Marks**		

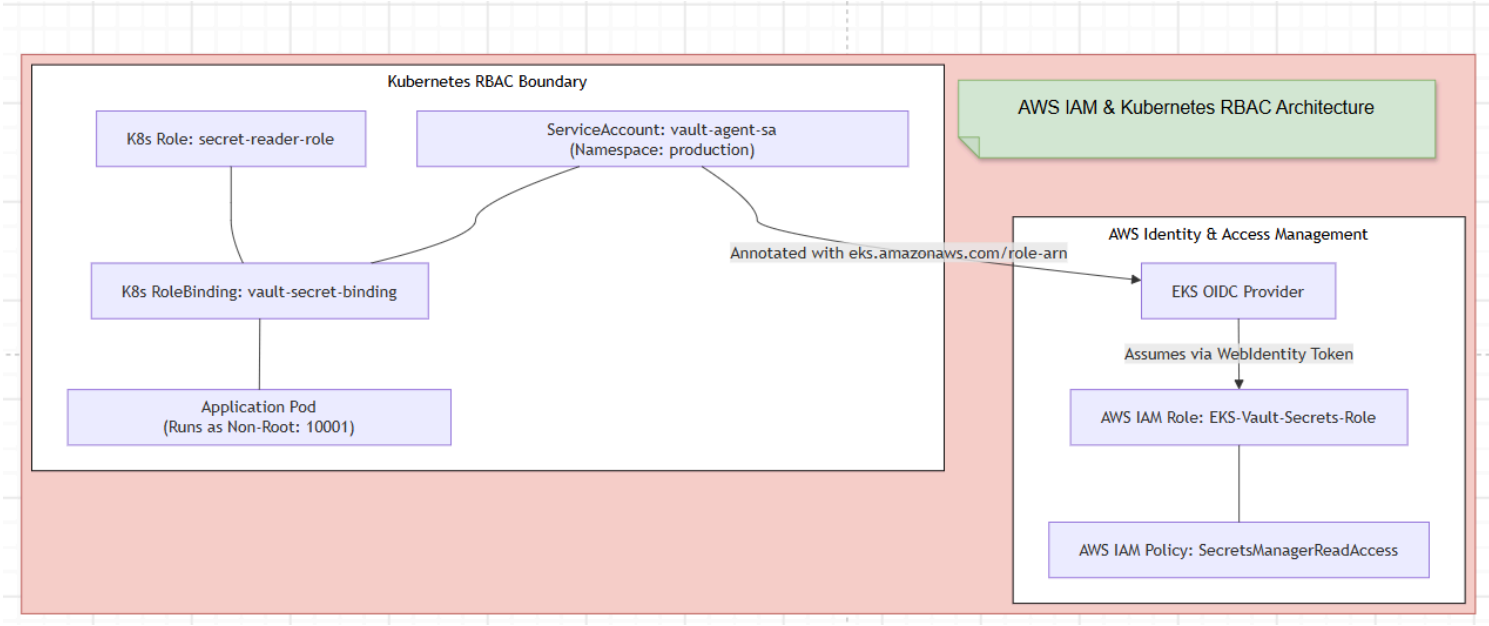
100



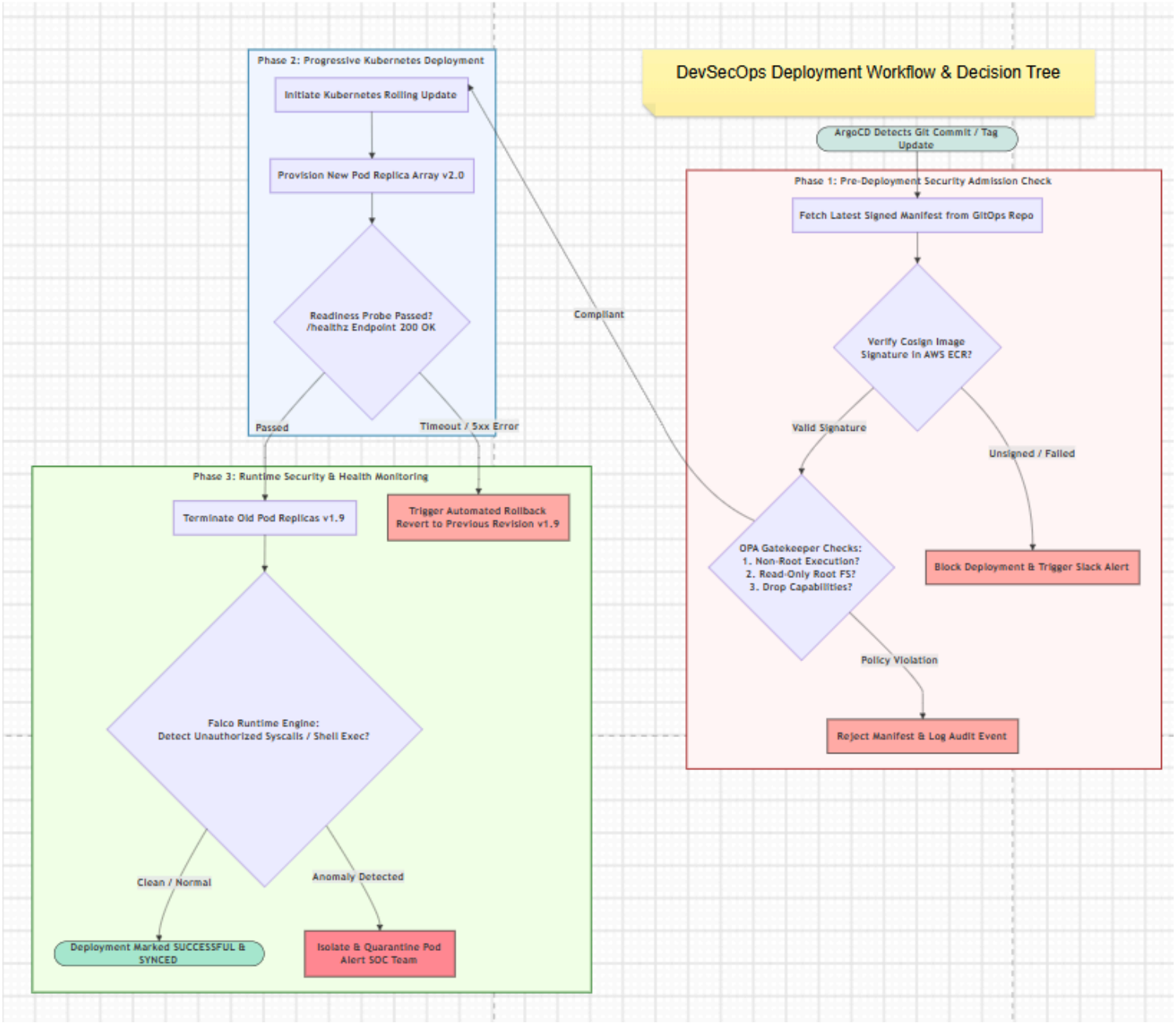
ARCHITECTURE DIAGRAM



GIT OPS SYNC WORKFLOW



AWS IAM AND KUBERNETS RBAC ARCHITECTURE



DEVSECOPS DEPLOYMENT WORKFLOW AND DECISION TREE

1. System Overview and Methodology

This project presents a zero-trust DevSecOps infrastructure designed for containerized cloud application workloads running on Amazon Elastic Kubernetes Service (AWS EKS). The primary goal of this architecture is to embed automated security controls into every stage of the Software Development Lifecycle (SDLC)—shifting security left during code submission and enforcing declarative security guardrails at runtime.

STRIDE Threat Modeling Framework Implementation

To systematically address security risks across trust boundaries, the infrastructure applies the STRIDE threat model:

- Spoofing Prevention: Developer identities are verified using mandatory GPG-signed Git commits and SSH key authentication. Continuous Integration (CI) runners authenticate to AWS infrastructure using temporary OpenID Connect (OIDC) identity tokens, eliminating static, long-lived access keys.
- Tampering Prevention: All container images built in the pipeline are cryptographically signed using Cosign before storage in Amazon Elastic Container Registry (ECR). ECR repositories enforce image tag immutability to prevent image overwriting. GitOps continuously audits cluster state to automatically revert unauthorized drift.
- Repudiation Prevention: Audit logging is enabled for the AWS EKS control plane (API server, audit log, authenticator, scheduler) and routed directly to AWS CloudWatch with log file integrity validation enabled.
- Information Disclosure Prevention: Sensitive operational data and Kubernetes Secrets are encrypted at rest using dedicated AWS Key Management Service (KMS) Customer Managed Keys. Secrets are injected at runtime via IAM Roles for Service Accounts (IRSA), eliminating hardcoded secrets.
- Denial of Service Prevention: Kubernetes namespace resource quotas, CPU/memory limits, and readiness/liveness probes prevent single workload failures from degrading cluster availability.
- Elevation of Privilege Prevention: Containers run as unprivileged users (UID 10001), use read-only root filesystems, drop all standard Linux capabilities, and are verified by Open Gate Agent (OPA) Gatekeeper admission controllers.

2. Security Control Mapping (NIST SP 800-53 Matrix)

The architectural controls map directly to NIST SP 800-53 enterprise compliance standards:

- Access Control (AC-6 - Least Privilege): Enforced via AWS IAM Roles for Service Accounts (IRSA) and fine-grained Kubernetes Role-Based Access Control (RBAC) RoleBindings.
- Configuration Management (CM-2 - Baseline Configuration): Achieved using Terraform Infrastructure as Code (IaC) for initial cluster provisioning and ArgoCD for maintaining state alignment via version-controlled Helm charts.
- System and Information Integrity (SI-7 - Software Integrity): Guaranteed by automated pre-commit SAST (SonarQube), SCA (Snyk), IaC (Checkov), container scanning (Trivy), and Cosign binary image signing.
- Protection of Information at Rest (SC-28): Executed using AWS KMS envelope encryption for cluster secrets, EBS storage volumes, and ECR artifact repositories.
- Information System Monitoring (SI-4): Handled in real time by the Falco runtime security engine monitoring Linux kernel syscalls for unauthorized process execution.

3. End-to-End GitOps Deployment Workflow

The continuous integration and continuous deployment (CI/CD) framework strictly decouples the Build Phase from the Deployment Phase using a pull-based GitOps pattern managed by ArgoCD.

Workflow Execution Stages:

1. Developer Commit & Trigger: A developer pushes GPG-signed code to a protected branch in the application repository. A webhook triggers the GitHub Actions CI pipeline.
2. Shift-Left Security Verification: The pipeline sequentially runs Static Application Security Testing (SAST), Software Composition Analysis (SCA) for third-party dependencies, and Infrastructure as Code (IaC) policy validation.
3. Container Build and Signing: If all security gates pass, the container image is compiled, scanned for vulnerabilities using Trivy, signed with Cosign, and pushed to Amazon ECR.
4. GitOps Manifest Update: The CI pipeline creates an automated pull request against the GitOps configuration repository, updating the target deployment image tag in the Helm values.yaml file.
5. Admission Control Verification: ArgoCD detects the change and attempts reconciliation. The deployment manifest is evaluated by the OPA Gatekeeper Admission Controller against internal security constraints (prohibiting root users, privilege escalation, and writeable filesystems).
6. Progressive Rolling Update: Once approved by OPA Gatekeeper, Kubernetes executes a rolling update deployment (maxSurge: 25%, maxUnavailable: 0%).
7. Automated Rollback & Monitoring: If the application readiness probes fail (/healthz endpoint returning non-200 responses), ArgoCD halts synchronization and automatically rolls back the cluster to the last known stable Git commit hash. Active pods are continuously monitored by Falco for kernel-level anomalies.

4. Identity and Access Management (IAM) Relationships

To enforce strict Least Privilege access within the EKS cluster, long-lived AWS credentials are replaced by OpenID Connect (OIDC) identity federation using IAM Roles for Service Accounts (IRSA).

Access Flow Mechanism:

- An AWS IAM Role (EKS-Vault-Secrets-Role) is assigned a minimal AWS IAM policy allowing secretsmanager:GetSecretValue strictly on designated application paths.
- A trust relationship (sts:AssumeRoleWithWebIdentity) is established between the EKS cluster's OIDC Provider and the IAM Role.
- A native Kubernetes ServiceAccount (app-service-account) in the production namespace is annotated with the target AWS IAM Role Amazon Resource Name (ARN).
- When a Pod launches under this ServiceAccount, the EKS Pod Identity Webhook automatically injects a short-lived JSON Web Token (JWT). The workload exchanges this token with the AWS STS service to fetch temporary AWS credentials, ensuring zero persistent credentials exist within the application container or cluster environment.

5. Artifact & Repository Deliverables Index

To support complete project maintainability, all source files are organized within the following version-controlled repository layout:

- /documentation: Contains the STRIDE Threat Modeling analysis and NIST SP 800-53 control compliance matrices.
- /diagrams: Stores raw editable source files for all architecture models, including DiagrammeR (.R) scripts and Mermaid (.mmd) visual flows.
- /iac: Holds executable HashiCorp Terraform configuration modules (main.tf, opa_constraint.tf, variables.tf) defining EKS KMS key rotation, private VPC subnets, cluster endpoints, and Gatekeeper policy rules.
- /gitops: Contains ArgoCD Application controllers (argocd-application.yaml) and production workload deployment manifests (deployment.yaml) configured with strict security contexts.